TRUST XAI: Model-Agnostic Explanations for AI With a Case Study on IIoT Security

Maede Zolanvari, *Student Member, IEEE*, Zebo Yang, *Student Member, IEEE*, Khaled Khan, *Senior Member, IEEE*, Raj Jain, *Life Fellow, IEEE*, and Nader Meskin, *Senior Member, IEEE*

***Abstract*—Despite artificial intelligence (AI)'s significant growth, its "black box" nature creates challenges in generating adequate trust. Thus, it is seldom utilized as a standalone unit in IoT high-risk applications, such as critical industrial infrastructures, medical systems, financial applications, etc. Explainable AI (XAI) has emerged to help with this problem. However, designing appropriately fast and accurate XAI is still challenging, especially in numerical applications. Here, we propose a universal XAI model, named the transparency relying upon statistical theory (TRUST), which is model-agnostic, high performing, and suitable for numerical applications. Simply put, TRUST XAI models the statistical behavior of the AI's outputs in an AI-based system. Factor analysis is used to transform the input features into a new set of latent variables. We use mutual information (MI) to rank these variables and pick only the most influential ones on the AI's outputs and call them "representatives" of the classes. Then, we use multimodal Gaussian (MMG) distributions to determine the likelihood of any new sample belonging to each class. We demonstrate the effectiveness of TRUST in a case study on cybersecurity of the Industrial Internet of Things (IIoT) using three different cybersecurity data sets. As IIoT is a prominent application that deals with numerical data. The results show that TRUST XAI provides explanations for new random samples with an average success rate of 98%. Compared with local interpretable model-agnostic explanations (LIME), a popular XAI model, TRUST is shown to be superior in the context of performance, speed, and the method of explainability. In the end, we also show how TRUST is explained to the user.**

***Index Terms*—Artificial intelligence (AI), cybersecurity, explainable AI (XAI), Industrial Internet of Things (IIoT), machine learning (ML), statistical modeling, trustworthy AI.**

I. Introduction

THE IMPACT of artificial intelligence (AI) on today's technological advancements is undeniable. Its applications range from daily life events such as medical decision support to fundamental demands such as cybersecurity. Despite the popularity of AI, it is limited by its current inability to build trust. Researchers and industrial leaders have a hard time explaining the decisions that sophisticated AI algorithms come up with because they (as AI users) cannot fully understand why and how these "black boxes" make their decisions.

Trusting AI blindly impacts its applicability and legitimacy. Based on Gartner's predictions, 85% of the AI projects until 2022 will produce inaccurate outcomes resulting from the organization's limited knowledge of the deployed AI and its behavior [1]. Also, the increasing growth in implementing AI has raised concerns of 91% of cybersecurity professionals about cyberattacks using AI [2]. In the medical field, dedicated AI algorithms are able to precisely (sometimes even more accurately than a human practitioner) detect rare diseases [3]. However, due to the lack of AI's decision transparency and explainability, the results are not considered trustworthy enough to be utilized in practice [4]. A global survey shows that more than 67% of the business leaders do not trust AI, and they believe AI's ambiguity will have negative impacts on their business in the next five years [5].

Moreover, the Internet of Things (IoT), as an integral part of both personal lives and industrial environments, has profited from AI for AI-powered data analysis. It allows connections, interactions, and data exchange among machines and devices for comprehensive functionality and higher efficiency without requiring any human in the loop. This automation requires a tremendous amount of trust while utilizing AI. This fact emphasizes why incorporating Explainable AI (XAI) in an AI-based IoT system is essential.

Designing self-explanatory models has gained much attention recently. XAI deals with psychology and cognitive science to provide transparent reasonings for AI's actions [6]. Based on how the underlying AI is designed and the input types (image, text, voice, and numerical data), the methods of explainability differ.

A proper XAI model should be integrated into the system to gain trust and transparency for AI-based systems. For clarity, in this article, we call the main underlying AI model the *primary AI model* and the explainable model that accompanies it, the *XAI model*. The primary AI model is in charge of the main functionality of the system, such as classification, regression, recommendation, recognition, etc. The XAI model is in charge of providing transparency and explanation in the primary AI's behavior.

The primary AI and the XAI could be both in one model, which we can interpret the model's outcomes directly using

Manuscript received 3 May 2021; revised 24 September 2021; accepted 15 October 2021. Date of publication 21 October 2021; date of current version 6 February 2023. This work was supported in part by the Qatar National Research Fund (QNRF) under Grant NPRP10-0206-170360, and in part by NSF under Grant CNS-1718929. *(Corresponding author: Maede Zolanvari.)*

Maede Zolanvari, Zebo Yang, and Raj Jain are with the Department of Computer Science and Engineering, Washington University, St. Louis, MO 63130 USA (e-mail: maede.zolanvari@wustl.edu; zebo@wustl.edu; jain@wustl.edu).

Khaled Khan and Nader Meskin are with the Department of Computer Science and Engineering, Qatar University, Doha, Qatar (e-mail: k.khan@qu.edu.qa; nader.meskin@qu.edu.qa).

Digital Object Identifier 10.1109/JIOT.2021.3122019

2327-4662 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:16:13 UTC from IEEE Xplore. Restrictions apply.

the model itself. This is achievable by using interpretable models as the primary AI model (e.g., decision tree, linear regression, etc.). Even though this type of XAI might provide potentially more accurate explanations, it is limited by the nonreusability, and we can only count on the model providers [7]. Also, learning performance and explainability of learning models usually have an inverse relationship [4]. Better learning performance comes with more sophistication in finding complex relationships in the data features. As such, it becomes more challenging for individuals to grasp its rationality. Therefore, in some applications, choosing interpretable models as the primary AI might yield a significantly degraded learning performance. Hence, surrogate explainers[1] running in parallel along with the primary AI have become preferable in the explanation domain. They are independent and do not affect or put restrictions on the performance of the primary AI model.

It is important to note that, in the XAI domain, it is not expected that the explanation would be in layman's terms. Most of the developed XAI models in the literature are just another interpretable AI or machine learning (ML) model that is easily understood. Therefore, it is assumed that the user is an expert in the internal workings of the learning model utilized as the XAI and knows the mathematics behind it. Here, we argue that statistics are significantly more verifiable and explainable than the AI or ML models used as the XAI. Therefore, we propose using statistical models to simplify the explanation process.

This article introduces an XAI model that provides transparency, relying upon statistical theory (TRUST). TRUST XAI is a surrogate explainer that provides interpretability without sacrificing the performance of the primary model or imposing restrictions on it. The explainer does not depend on the type of the primary AI in any way; thus, it is entirely model-agnostic.[2] We believe that TRUST XAI will complement the future AI deployments as a standalone unit by providing transparent, independent, and reliable explanations.

While many papers in the XAI domain have focused on image-based data [7], [8], TRUST XAI applies to numerical data such as network data for IoT and security systems, which is critical but currently deficient in XAI.

The TRUST technique works solely with the feature sets and the outputs from the primary AI. If the primary AI is modeled as shown in Fig. 1 on the left side, TRUST XAI is represented by the three steps on the right using a background with diagonal stripes. The first step builds the core of the explainer, and it has to be calculated just once. Then, only the last two steps are needed for explaining any new data instances.

To evaluate and provide a practical instance for TRUST XAI, we use three data sets: 1) "WUSTL-IIoT," a data set generated from our prior work on our industrial IoT (IIoT) testbed [9]; 2) "NSL-KDD," a public data set for network-based intrusion detection system (IDSs) from the University of New Brunswick [10]; and 3) "UNSW," another IDS data set from the Cyber Range Lab of the Australian Centre for Cyber Security (ACCS) [11]. We empirically prove that our proposed XAI model successfully explains the labels predicted by the primary AI in just a few milliseconds. Moreover, we have implemented and applied local interpretable model-agnostic explanations (LIME) on the three mentioned data sets for the sake of comparison. Our results show LIME is much slower than our model and is not practical for real-time applications or when dealing with a large number of samples.

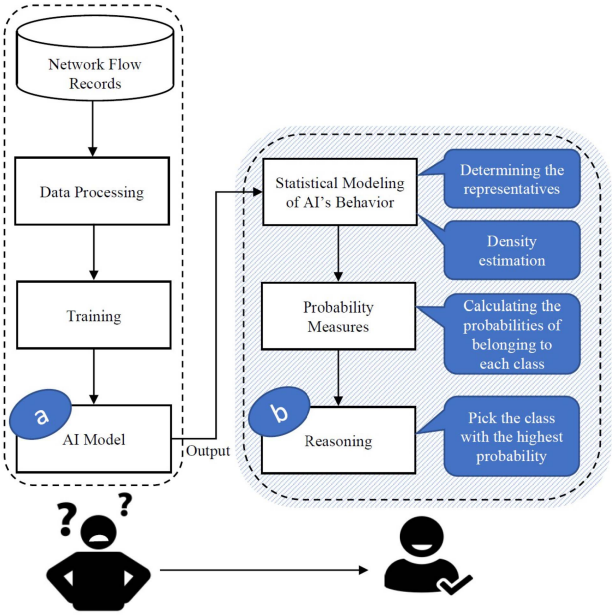


Fig. 1. Integrated TRUST explainer in an AI-based system. While (a) can be very complex and noninterpretable, (b) should be as easily interpretable as possible for humans.

The research contributions of this work are as follows.

1) We propose a universal, accurate, and well-performing XAI model called TRUST XAI that requires no compromise in the selection or performance of the primary AI, while at the same time providing transparency in the model's results.
2) We show that TRUST XAI is very fast at reasoning the primary AI model's behavior on numerical data, making it one of the first XAI models suitable for real-time numerical applications.
3) Our case study introduces a unique perspective on the assessment of AI-based trustworthy systems for IIoT and Industry 4.0 [12].
4) Our IIoT security data set[3] would be released to support the research community for a more extensive and more diverse data collection in the emerging field of AI and XAI for IIoT.

As data privacy and integrity are essential in all areas of IoT, cybersecurity is of paramount importance. Utilizing AI to provide a secure platform in this area has been proven as an effective method, especially in mission-critical applications

[1]This means developing a separate model working in parallel with the primary AI model to explain its behavior.

[2]This simply means the XAI does not care about the primary AI algorithm. All it requires is being able to probe the model with any arbitrary input and get an output from the "black box."

[3]https://www.cse.wustl.edu/~jain/iiot2/index.html

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:16:13 UTC from IEEE Xplore. Restrictions apply.

such as IIoT [9], [13], [14]. Our case study on the security of IIoT indicates our belief in the importance of IIoT and Industry 4.0. However, TRUST XAI is universal and can be applied to any other applications with minor or zero modification.

The remainder of the article is organized as follows. The related research papers are explored in Section II. In Section III, we discussed the proposed TRUST model, including how we formalize the problem, what the assumptions are, and how we develop a proper model. Following that, to test TRUST, we test its performance through a case study in Section IV. How TRUST would interpret the results to the user is explained in Section V. Finally, Section V concludes the article, while mentioning the limitations of TRUST and future work.

II. Related Work

The application of XAI in the IoT domain has emerged as a substantial promise for improving trust and transparency. Examples include industrial settings [15], [16], smart grids [17], 5G telecommunication [18], smart homes [19], and healthcare [20], [21], along with XAI surveys covering different other areas, such as [7], [22]–[25], and [26]. XAI has been recognized as a necessary trend toward the next generation of AI-based systems, aiming to produce transparent models without affecting the primary AI's accuracy. The tradeoffs between accuracy, interpretability, reusability, and performance have been continuously studied [3], [6], [7], [22].

As discussed in [27], feature importance is the most commonly utilized method of explainability. It is the basic technique to find the key features impacting the AI's outcome. Feature selection is a common tool in data preprocessing [28]. It discovers important features that affect the AI's output in general across all classes. However, as seen in the following works, pinpointing these features alone cannot provide a baseline for evaluating or comparing the performance of the proposed XAI.

For example, in [29], the Shapley additive explanations (SHAP) value, which is a feature importance technique originally developed by Lundberg and Lee [30], are used to explain the predictions made by the primary AI model in network security. The results of the XAI model are presented as the list of features that are more indicative of the class of data. However, even the authors confirmed that SHAP is computationally heavy and not suitable for real-time applications. In [31], another feature importance technique, layerwise relevance propagation (LRP), is developed and specialized for the explainability of deep neural network (DNN) models. It should potentially provide accurate explanations for DNN, but it cannot be generalized for other models.

In [32], modeling a random forest with a decision tree has been proposed to add interpretability to the results. We know that while decision trees are highly explainable, random forest suffers from being complex and lack in interpretability. However, this technique is not model-agnostic since it is only applicable in tree-based learners. In [33], an adversarial technique is used as a means of interpretability. The method calculates the minimum adjustments to the important features' values of a misclassified sample until the primary AI correctly predicts it. However, the proposed method is semi-agnostic since it works only for the AI models with a defined gradient cross-entropy loss function. Additionally, no performance assessment is provided.

Finally, LIME, which is used as a *benchmark XAI* in this article, has been a popular technique in the XAI domain, which has many derivatives since its introduction in [34], such as [35] and [36]. It generates local surrogate models for every instance to explain its label. For each instance, a new data set is synthesized consisting of samples with feature values drawn from a normal distribution with a mean and standard deviation corresponding to that instance. The primary AI then labels the synthesized samples. Also, they are assigned weights based on their distance from the instance of interest. Next, a simple interpretable AI or ML model (usually, a linear regression model) is trained with the new synthetic data set to explain the outcome of the primary AI for that specific instance. LIME is re-evaluated and compared with Anchors introduced in [37] by the same authors. Their results show that a linear explanation like LIME is preferred in cases where predictions are near the primary AI's decision boundary because linear explanations provide insight into the primary AI, while Anchors contributes by providing the coverage of the explanations with high accuracy (e.g., when the word "not" has a positive influence on the sentiment, such as "not bad"). Later, in this article, we compare the performance of LIME with our TRUST model and show how our model outperforms LIME in terms of speed and performance.

III. Proposed TRUST Explainer

The goal of TRUST XAI is to explain the rationale behind the data labeling of the primary AI model by applying statistical techniques to the AI's outputs. Table I presents the symbols used in this article. Symbols that are explained along with the equations or only used once are skipped from this table due to length considerations.

A. Problem Formalization

To formalize the problem that the primary AI is solving, TRUST XAI produces a set of vectors called "representatives" from the features of the training set. We make three assumptions about the data set and the representatives as follows.

Assumption 1: The statistical inferences made by the XAI model must represent the data's characteristics. Therefore, the data used to train the primary AI are also used to build the TRUST model. If the primary AI model has to be retrained due to a significant change in the data's characteristics, the core of the explainer must be rebuilt as well.

Assumption 2: The representatives are mutually independent. This allows their joint probability function to be the multiplication of individual probability functions. This assumption simplifies our XAI technique and makes it easier for human users to understand. We select representatives so that the correlation among them is close to zero.

Assumption 3: Assume that for the ith representative R_i, we have a sample $\boldsymbol{r}_i^c$ with N_c observations belonging to the

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:16:13 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SYMBOL TABLE

Symbol	Description
N	Number of observations in the training set
C	Total number of classes
c	Index indicating class number, $c \in \{1, ..., C\}$
N_c	Number of observations classified by the primary AI as being in class c, $\sum_{c=1}^{C} N_c = N$
K	Number of features used in the primary AI model
k	Number of representatives used in the XAI model, $k \ll K$
i	Index indicating the feature number $i \in \{1, ..., K\}$ for the primary AI model or the representative number $i \in \{1, ..., k\}$ for the XAI model
f_i	i^{th} feature
$\mathcal{F}$	The feature set: $\{f_1, ..., f_K\}$
F_i^c	Random variable of ith factor for class c, $F_i^c \in \mathbb{R}^{N_c \times 1}$
F_i	The ith factor; vertical concatenation of factors $\{F_i^1, \cdots, F_i^C\}$
R_i^c	Random variable of ith representative for class c, $R_i^c \in \mathbb{R}^{N_c \times 1}$
R_i	The ith representative; vertical concatenation of $\{R_i^1, \cdots, R_i^C\}$
$r_{j,i}^c$	R_i^c value in the jth observation, $j \in \{1, ..., N_c\}$
$\boldsymbol{r_i^c}$	Vector of observations belonging to the random variable R_i^c
$\boldsymbol{R^c}$	$N_c \times k$ observation matrix consisting of $r_{j,i}^c$ values arranged so that the column i consists of $\boldsymbol{r_i^c}$'s values
M_i^c	Number of modes of R_i^c in its MMG distribution
w_i	Importance coefficient of R_i
λ_i^c	Eigenvalue of F_i^c

class c. The distribution of these observations can be approximated with reasonable accuracy by a 1-D multimodal Gaussian (MMG) distribution with M_i^c subpopulations, which are called *modes*, with $\{\mu_{i,1}^c, \ldots, \mu_{i,M_i^c}^c\}$ means and $\{\sigma_{i,1}^c, \ldots, \sigma_{i,M_i^c}^c\}$ standard deviations. In the case of a unimodal distribution, M_i^c is equal to one.

B. Determining the Representatives

Since factor analysis [38] satisfies the requirements mentioned in Assumption 2 and more, we use it to determine the representatives. It provides a linear combination of the data set's features. It also reduces the redundancy in the feature space, which makes our model less likely to overfit. Furthermore, since these new independent latent variables are combinations of the features, we still preserve the most valuable parts of all features. Factor analysis projects the feature values onto axes that maximize the percentage of explained variance in the data. Another advantage of this technique is that the resulting factors are orthogonal. Therefore, they are independent with zero covariations.

More specifically, we use the factor analysis of mixed data (FAMD) that takes into account different types of variables (quantitative and qualitative) in a data set [38]. FAMD is considered the general form of factor analysis. Generally speaking, the core of FAMD is based on principal component analysis (PCA) when variables are all quantitative and multiple correspondence analysis (MCA) when all the variables are qualitative.

When dealing with a mixed data set, the typical approach in the research community is either neglecting or encoding (e.g., one-hot encoding) the categorical data when using PCA or transforming the quantitative variables into qualitative variables by breaking down their variation intervals into different levels when using MCA. Despite being relatively easy to implement, these approaches are not accurate enough, and we lose lots of information from the data. Therefore, FAMD is considered the most accurate form of analysis when dealing with a mixed data set because it takes into account both types of data [39].

Assume that we have a feature set $\mathcal{F} = \{f_1, \ldots, f_K\}$ of mixed data, where K is the total number of the features. After applying factor analysis to the observations belonging to each class separately, we have K factors determined for each class. Note that the number of features and factors are both K. For example, for class c, factors $\{F_1^c, \ldots, F_K^c\}$ are produced. The pseudocode of this function, `factorAnalysis`, is presented as Algorithm 1. In this algorithm, each data instance belongs to one of the C classes.

Algorithm 1 Factor Analysis

Function: `factorAnalysis`
Input: Training Set: $X \neq \emptyset \in \mathbb{R}^{N \times K}$;
AI Model: $\mathcal{AI}$; Features Set: $\mathcal{F}$
$y \leftarrow \mathcal{AI}(X)$
Set labels of each row in $X \leftarrow y$
Divide X into $X^c \in \mathbb{R}^{N_c \times K}$ sets per class
for each X^c **do**
  $X^{c\prime} \leftarrow$ standardized (X^c) ▷ So it has zero mean and unit variance
  Build the relationship matrix (RM^c) such that:
  for each pair (i, j) in $\mathcal{F}(X^c)$ **do**
    if f_i & f_j are both quantitative **then**
      $RM_{i,j}^c = \rho_{f_i,f_j}^2$
    else if f_i & f_j are both qualitative **then**
      $RM_{i,j}^c = \chi_{f_i,f_j}^2$
    else
      $RM_{i,j}^c = \eta_{f_i,f_j}^2$
    end if
  end for
  $\{F_1^{c\prime}, \ldots, F_K^{c\prime}\}$ & $\{\lambda_1^c, \ldots, \lambda_K^c\} \leftarrow$ SVD(RM^c)
  $\{F_1^c, \ldots, F_K^c\} \leftarrow$ unstandardized $(\{F_1^{c\prime}, \ldots, F_K^{c\prime}\})$
  return $\{F_1^c, \ldots, F_K^c\}$
end for

In Algorithm 1, ρ_{f_i,f_j}^2 is the Pearson correlation coefficient, χ_{f_i,f_j}^2 is the chi-square, and η_{f_i,f_j}^2 is the squared correlation ratio between feature i and feature j from the standardized matrix of X^c [40], [41]. When measuring the relationship between two quantitative variables, the Pearson correlation coefficient is used, and for two qualitative variables, chi-square is used. Furthermore, when dealing with one variable from each kind, we use the general form of the squared correlation ratio, which results in the Pearson correlation coefficient if both variables are quantitative or chi-square if they are both qualitative. Simply put, these three methods all boil down to the same result, measuring correlations between each pair of variables building a relation matrix for each class. Finally, singular value decomposition (SVD) is applied to the relation matrix

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:16:13 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Picking Representatives

Function: `pickingReps`
Input: Training Set: $X \neq \emptyset \in \mathbb{R}^{N \times K}$;
AI Model: $\mathcal{AI}$; Features Set: $\mathcal{F}$
1: $\{F_1^c, \ldots, F_K^c\} \leftarrow$ `factorAnalysis`$(X, \mathcal{AI}, \mathcal{F})$
2: $y \leftarrow \mathcal{AI}(X)$
3: **for** each i **do**
4: $F_i = [{F_i^1}^T, \cdots, {F_i^C}^T]^T$
5: $w_i \leftarrow \text{MI}(y, F_i)$
6: ${w_i}' \leftarrow$ sort w_i in descending order
7: $\{R_1^c, \ldots, R_k^c\} \leftarrow$ Pick top k factors with the highest ${w_i}'$s
8: **return** $\{R_1^c, \ldots, R_k^c\}$
9: **end for**

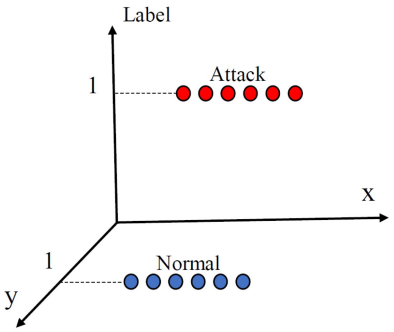


Fig. 2. 2-D data with features highly correlated with the labels.

RM^c for factorization, removing the interdependencies among the features and projecting them in orthogonal dimensions.

Then, in Algorithm 2, we use the factors calculated from Algorithm 1 to pick the "representatives" based on how important each factor is.

Factor analysis (Algorithm 1) produces an eigenvalue λ_i^c for each factor F_i^c, which is equivalent to the variation in the data that the factor explains. However, the effectiveness of a factor in distinguishing the class label is not related to the percentage of the explained variation of the data by that factor. Therefore, the percentages of explained variation cannot be used as the importance coefficients for the factors. We elaborate more on this in the following.

Assume that we have 2-D data with their class labels on the third axis. An example is demonstrated in Fig. 2. The attack class is shown by the red points and the normal class by the blue points. As seen in this figure, since the variation in the y-axis values per class is not large, the y-axis feature would not be considered important. Therefore, in the case of a dimension reduction or factor analysis, separately for each class, the x-axis values would be the primary axis representing each class's samples. This is because the x-axis explains almost all the variations in the data points for each class. Meanwhile, it is trivial to see that their y-axis values could easily distinguish the class labels. Hence, removing them or treating them as not important will cause a significant loss in the accuracy of the proposed model.

Therefore, we devised a method using the correlation between each factor and the class label to rank the factors and pick the ones with the highest correlation. We use mutual information (MI), which quantifies the amount of information we can get about one variable by knowing the other. It measures the mutual dependence between two variables, which, in our case, are the factors and the class labels. MI between the factors and the class labels is calculated as

$$\text{MI}(y, F_i) = \text{H}(y) - \text{H}(y|F_i) \tag{1}$$

where $F_i = [{F_i^1}^T \cdots {F_i^C}^T]^T$ is the vertical concatenation of the ith factor values for all classes. The T superscript denotes a transpose of the vector/matrix. $\text{MI}(y, F_i)$ is the MI between the class label vectors y and F_i. H(.) is the entropy function, which is defined as follows:

$$\text{H}(y) = -\sum_c P(y = c) \log(P(y = c)). \tag{2}$$

For the sake of simplicity and not having to use differential entropy (since F_i is not discrete), we bin the values in F_i, and ζ_i's are the possible outcomes of the binned random variable F_i. Assume that $P(\zeta_i)$ is the probability of $F_i = \zeta_i$. The conditional entropy $\text{H}(y|F_i)$ is calculated as follows:

$$\begin{aligned}\text{H}(y|F_i) &= \sum_{\zeta_i} P(F_i = \zeta_i)\text{H}(y|F_i = \zeta_i) \\ &= -\sum_{\zeta_i} P(F_i = \zeta_i) \sum_c P(y = c|F_i = \zeta_i) \log P(y = c|F_i = \zeta_i) \\ &= -\sum_{\zeta_i} \sum_c P(c, \zeta_i) \log \frac{P(c, \zeta_i)}{P(\zeta_i)}.\end{aligned} \tag{3}$$

We use the MI values as the importance coefficients for the factors. For instance, w_i, the importance coefficient of F_i, is equal to $\text{MI}(y, F_i)$. For more details about MI, we refer to [42].

Then, we pick only the first k factors, $k \ll K$, with the highest w_i values as the "representatives." This means the representatives are the factors that have the highest correlation with the class labels. They represent each class's feature values and behavior.

There is no need to represent each class by its all K factors. Based on the application constraints, we can choose k. Regarding the choice of k (i.e., the number of representatives), there is a tradeoff between time consumption and performance. For real-time applications, we can pick a lower k (meaning a few factors with the highest importance coefficients as the representatives). For applications where high accuracy is important, we can go with a higher k. Since the density and mode computations take time, the more representatives we have, the slower the model would be, but at the same time, it leads to higher accuracy. The time consumption analysis is provided later in this article. The algorithm to select the representatives is summarized in Algorithm 2.

C. Density Estimation

To study the statistical attributes of the representatives' values, we approximate their density functions with 1-D MMG distributions.

Assume that for all the data instances of the training set labeled as the class c by the primary AI model, X^c, representative R_i^c $i = 1, \ldots, k$ has M_i^c modes. For now, assume that we know the value of M_i^c; we will discuss how to calculate

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:16:13 UTC from IEEE Xplore. Restrictions apply.

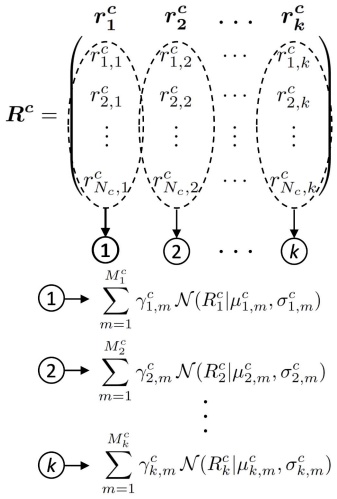


Fig. 3. Approximating representatives to unique MMG distributions for class c.

it later in the next section. The 1-D multimodal probability density functions (pdfs) of R_i^c can be modeled as follows:

$$p_i(R_i^c) = \sum_{m=1}^{M_i^c} \gamma_{i,m}^c \mathcal{N}\big(R_i^c | \mu_{i,m}^c, \sigma_{i,m}^c\big). \tag{4}$$

$\gamma_{i,m}^c$ is the component weight of subpopulation m for R_i^c observations in class c. $\mathcal{N}$ is a normal distribution with mean $\mu_{i,m}^c$ and standard deviation $\sigma_{i,m}^c$. To make the total area under the pdf normalized for every representative per class, for $\forall c \vee \forall i$, $\sum_m \gamma_{i,m}^c = 1$ should be satisfied. Here, we use the expectation–maximization (EM) algorithm [43] to fit the representatives' values to a proper distribution in the form of (4).

After the above process, we end up with $k \times C$ distributions. These distributions build the backbone of the TRUST explainer. Fig. 3 shows the approximation process for the values of class c representatives as an example. $\boldsymbol{R^c} = [\boldsymbol{r_1^c}, \boldsymbol{r_2^c}, \ldots, \boldsymbol{r_k^c}] \in \mathbb{R}^{N_c \times k}$ equates to the matrix of observations of the representatives of the class c, where R_i^c is the random variable for the set of observations $\boldsymbol{r_i^c}$.

Now, it is time to show how our TRUST XAI explains the data instances labeled as a specific class by the primary AI model. First, the likelihoods that the new instance's representatives belong to each class's representatives are calculated. These likelihoods can be estimated from the pdf models. The total likelihood is the product of the likelihoods of all representatives. We apply a logarithm transformation to decrease the computational complexity and compute the log likelihoods. Thus, we simplify the computations from multiplications to additions (which is less expensive), avoid computation of the exponentials, and prevent any arithmetic underflow or overflow (since likelihoods could become very large or small). Log transformation also reduces or removes the skewness in the data. Assume that the new instance is $\boldsymbol{z} = [z_1, \ldots, z_K] \in \mathbb{R}^{1 \times K}$. The inputs to TRUST XAI are only the representatives' values. Therefore, $\boldsymbol{z}$ is projected on the factors' space and cropped to its corresponding representatives' values $\boldsymbol{z'} = [z'_1, \ldots, z'_k] \in \mathbb{R}^{1 \times k}$. The conditional likelihood of z'_i, $i = 1, \ldots, k$ belonging to class c can be written as

$$p_i(z'_i|c) = \sum_{m=1}^{M_i^c} \exp\big(\log(\gamma_{i,m}^c) + \log\big(\mathcal{N}\big(z'_i | \mu_{i,m}^c, \sigma_{i,m}^c\big)\big)\big). \tag{5}$$

After taking a logarithm from (5), the conditional log likelihood of z'_i belonging to R_i^c is calculated by (6)

$$\begin{aligned} &\log(p(z'_i|c)) \\ &= \log\left(\sum_{m=1}^{M_i^c} \exp\big(\log(\gamma_{i,m}^c) + \log\big(\mathcal{N}\big(z'_i | \mu_{i,m}^c, \sigma_{i,m}^c\big)\big)\big)\right) \\ &= \log\left(\sum_{m=1}^{M_i^c} \exp\left(\alpha_{i,m}^c - \frac{1}{2}\left(\frac{z'_i - \mu_{i,m}^c}{\sigma_{i,m}^c}\right)^2\right)\right) \end{aligned} \tag{6}$$

where

$$\alpha_{i,m}^c = \log(\gamma_{i,m}^c) - \log(\sigma_{i,m}^c) - \frac{1}{2}\log(2\pi). \tag{7}$$

$\alpha_{i,m}^c$ $\forall c \vee \forall i \vee \forall m$ is a fixed constant.

Subsequently, we calculate the weighted sum of these log likelihoods using the representatives' importance coefficients (the MI values, w_i, from Section B). The w_i's are normalized before the calculation, though. The importance coefficient of the representative R_i is calculated by

$$\hat{w}_i = \frac{w_i}{\sum_{j=1}^k w_j}. \tag{8}$$

Since w_i's are normalized, $\sum_{i=1}^k \hat{w}_i = 1$. After that, the log likelihood of classifying $\boldsymbol{z'}$ as class c is computed as

$$\log(p(\boldsymbol{z'}|c)) = \sum_{i=1}^{k} \hat{w}_i \times \log(p_i(z'_i|c)). \tag{9}$$

Finally, the predicted class for z by the primary AI, represented as ℓ_z, is explained by TRUST XAI as the class with the maximum total log likelihood

$$\ell_z = \max_c \log(p(\boldsymbol{z'}|c)). \tag{10}$$

It is essential to notice that we are not following the common clustering processes that model the whole training set as a multidimensional (with features as the dimensions) multicluster distribution and then calculate the likelihood of the new data belonging to each cluster (i.e., class). Here, we take a completely different approach. We approximate the 1-D distribution of each representative and then calculate the likelihood of the new instance's representative values belonging to the corresponding representative distributions for each class. Afterward, by the weighted sum of these likelihoods for each class, we have C likelihoods, each resulting from k distributions. Then, the class with the highest likelihood is picked as the explained class.

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:16:13 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 Grid Mode Selection

Function: `modeSelect`
Input: Representatives of all the classes:
$\{R_1^1, \ldots, R_k^1\} \in \mathbb{R}^{N_1 \times k}, \cdots, \{R_1^C, \ldots, R_k^C\} \in \mathbb{R}^{N_C \times k}$;
Zone $Z \in \mathbb{R}^{C \times C}$
$y \leftarrow [\text{ones}(N_1)^T, \ldots, C \times \text{ones}(N_C)^T]^T$
for each set of $(R_i^1 \cdots R_i^C)$ **do**
  $Max \leftarrow 0$
  for m^1 in Z **do**
    $\text{MMG}_i^1 \leftarrow$ fit R_i^1 to a Gaussian with m^1 modes
    ⋮
    for m^C in Z **do**
      $\text{MMG}_i^C \leftarrow$ fit R_i^C to a Gaussian with m^C modes
      $P_{m^1} \leftarrow p_1(R_i|\text{MMG}_i^1)$
      ⋮ ▷$R_i = [R_i^{1^T}, \ldots, R_i^{C^T}]^T$
      $P_{m^C} \leftarrow p_C(R_i|\text{MMG}_i^C)$
      $P \leftarrow [P_{m^1} \cdots P_{m^C}]$
      $y_{\text{MMG}} \leftarrow \text{index}(P.\max(1))$
      $\text{score}_{m^1,\cdots,m^C} \leftarrow \text{MCC}(y_{\text{MMG}}, y)$
      if $\text{score}_{m^1,\cdots,m^C} > Max$ **then**
        $Max \leftarrow \text{score}_{m^1,\cdots,m^C}$
        $M_i^1 \leftarrow m^1$
        ⋮
        $M_i^C \leftarrow m^C$
      end if
    end for
    ⋮
  end for
end for
return $\{M_1^1, \ldots, M_k^1\} \cdots \{M_1^C, \ldots, M_k^C\}$

Algorithm 4 Fast Grid Search

Function: `modeSelectFast`
Input: Representatives of all the classes:
$\{R_1^1, \ldots, R_k^1\} \in \mathbb{R}^{N_1 \times k}, \cdots, \{R_1^C, \ldots, R_k^C\} \in \mathbb{R}^{N_C \times k}$;
Zone $Z \in \mathbb{R}^{C \times C}$
$y \leftarrow [\text{ones}(N_1)^T, \ldots, C \times \text{ones}(N_C)^T]^T$
for each set of $(R_i^1 \cdots R_i^C)$ **do**
  Divide zone Z into $z_1, \ldots, z_d$
  $Max \leftarrow 0$
  for each z_j **do**
    $(m^1, \cdots, m^C) \leftarrow z_j$'s center
    $\text{MMG}_i^1 \leftarrow$ fit R_i^1 to a Gaussian with m^1 modes
    ⋮
    $\text{MMG}_i^C \leftarrow$ fit R_i^C to a Gaussian with m^C modes
    $P_{m^1} \leftarrow p_1(R_i|\text{MMG}_i^1)$
    ⋮ ▷$R_i = [R_i^{1^T}, \ldots, R_i^{C^T}]^T$
    $P_{m^C} \leftarrow p_C(R_i|\text{MMG}_i^C)$
    $P \leftarrow [P_{m^1} \cdots P_{m^C}]$
    $y_{\text{MMG}} \leftarrow \text{index}(P.\max(1))$
    $\text{score}_{m^1,\cdots,m^C} \leftarrow \text{MCC}(y_{\text{MMG}}, y)$
    if $\text{score}_{m^1,\cdots,m^C} > Max$ **then**
      $Max \leftarrow \text{score}_{m^1,\cdots,m^C}$
      $Z_i \leftarrow z_j$
    end if
  end for
  `modeSelect`$(\{R_i^1\}, \cdots, \{R_i^C\}, Z_i)$
end for
return $\{M_1^1, \ldots, M_k^1\} \cdots \{M_1^C, \ldots, M_k^C\}$

D. Selecting the Number of Modes

A critical parameter in the TRUST explainer is the number of modes for MMG distribution (i.e., M_i^c) for each representative (as mentioned in Assumption 3). In general, this number is different for each representative, and it can be optimized to increase speed and performance. We use the grid search technique which is a standard method for determining hyperparameters in ML techniques. Through the grid search, for each representative, we find the number of modes that maximizes the probability of the correct prediction for samples from different classes. The choice of M_i^c for each representative is important since it has a critical role in how well the fitted MMG distribution works as a density estimator. The pseudo-code of the method is shown in Algorithm 3. This algorithm searches over a zone called Z, which is simply a grid of integers starting from 1 to any potential number of modes.

As seen in Algorithm 3, the dimension of the search space grows exponentially with the number of classes. Also, if the number of modes or classes is large, the search zone would be too large. Hence, the search could be very slow and time-consuming. We have developed a faster algorithm that divides the search zone into smaller subzones, as is shown in Algorithm 4. First, the probabilities of the centers of each subzone as the number of modes are calculated. Then, the subzone with the highest score is chosen to be searched thoroughly. Even though this algorithm might not give us the exact number of modes, it provides a good accuracy (in the order of 99% in our experiments). With the decrease in the size of subzones, the accuracy increases.

As seen in line 14 of Algorithm 3 and line 15 of Algorithm 4, Matthew's correlation coefficient (MCC), (11), is used to calculate the score. We have not used the Accuracy metric, (12), because MCC has been shown to be a better metric when dealing with imbalanced data sets, which are common in cybersecurity and other numerical applications [44]. The undetected rate (UR) (13) is another useful metric for performance evaluation used in the next section

$$\text{MCC} = \frac{\text{TP} \times \text{TN} - \text{FP} \times \text{FN}}{\sqrt{(\text{TP}+\text{FP})(\text{TP}+\text{FN})(\text{TN}+\text{FP})(\text{TN}+\text{FN})}} \tag{11}$$

$$\text{Accuracy} = \frac{\text{TP}+\text{TN}}{\text{TP}+\text{TN}+\text{FP}+\text{FN}} \tag{12}$$

$$\text{UR} = \frac{\text{FN}}{\text{FN}+\text{TP}} \tag{13}$$

where TN is the number of normal data labeled as normal, TP is the number of attack data classified as attack, FP is the number of normal labeled as attack, and FN is the number of attacks labeled as normal by the model.

IV. Case Study: IIoT Security

Sophisticated AI models have been vastly applied in IDSs. XAI would add interpretability and trust to these systems. We have built a lab-scaled IIoT system to collect realistic and up-to-date data sets for network-based IDSs. We have implemented a supervisory control and data acquisition (SCADA) system widely used by industries to supervise the level and turbidity of liquid storage tanks. This IIoT system is employed in industrial reservoirs and water distribution systems as a part

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:16:13 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SPECIFICS OF THE THREE DATA SETS

Dataset	WUSTL-IIoT	NSL-KDD	UNSW
# of observations	1,194,464	125,973	65,535
# of features	41	40	41
# of attacks	87,016	58,630	45,015
# of normals	1,107,448	67,343	20,520

of the water treatment and distribution. For more information regarding our testbed, we refer the readers to our previous papers [9] and [45].

A. Utilized Data Sets

The first tested data set is the one collected from our testbed, which we refer to as "WUSTL-IIoT." To collect the proper data set, we (as a white-hat attacker) attacked our testbed using manipulated commands, such as backdoor, command injection, denial of service, and reconnaissance [9].

Furthermore, we have tested our TRUST XAI on two other cybersecurity data sets to add an empirical proof of concept, "NSL-KDD" [10] and "UNSW" [11]. Specifications of these data sets are shown in Table II. NSL-KDD is a revised and cleansed version of the KDD'99 published by the Canadian Institute for Cybersecurity from the University of New Brunswick. KDD'99 was initially collected by the MIT Lincoln Laboratory through a competition to collect traffic records and build a network intrusion detector. This data set consists of four different types of attacks. Here, we label them all as class 1 for binary classification. Furthermore, the UNSW-NB15 data set, which we refer to as UNSW, was built by the Cyber Range Lab of the ACCS in 2015. The traffics are labeled as either 0 or 1 for a normal or attack class, respectively.

B. Primary AI Model

TRUST XAI is a model-agnostic technique that can be applied to any complex AI model. To evaluate its performance on IIoT security, we use an artificial neural network (ANN) as a binary classifier with two hidden layers with 20 and 10 neurons, respectively, with *ReLU* activation functions, ten epochs, with the *adam* optimizer, and *sigmoid* activation function in the output layer. The scikit-learn library [46] was used to implement the ANN model (as the primary AI model). ANN is generally considered complex and unexplainable to human users. The inputs to the IDS are the flow instances as mentioned in the previous section. The output of the IDS can be a multiclass or binary classification. However, for the sake of simplicity, we have built the TRUST explainer by treating the outputs of the IDS as binary classes with normal as 0 and attack as 1.

Even though the performance of the primary AI is not of our interest, we show its results on the three data sets in Tables III–V for comparison. Each data set is split into training and test sets with an 80:20 ratio. The performance on the training set is presented in the (a) tables, while the test results are shown in the (b) tables.

TABLE III
PERFORMANCE OF THE PRIMARY AI ON WUSTL-IIOT.
(a) ON THE TRAINING SET. (b) ON THE TEST SET

(a)

	Normal	Attack
Normal	885980	12
Attack	131	69448

(b)

	Normal	Attack
Normal	221452	4
Attack	40	17397

TABLE IV
PERFORMANCE OF THE PRIMARY AI ON NSL-KDD.
(a) ON THE TRAINING SET. (b) ON THE TEST SET

(a)

	Normal	Attack
Normal	53664	173
Attack	529	46412

(b)

	Normal	Attack
Normal	13452	54
Attack	138	11551

TABLE V
PERFORMANCE OF THE PRIMARY AI ON UNSW.
(a) ON THE TRAINING SET. (b) ON THE TEST SET

(a)

	Normal	Attack
Normal	15791	657
Attack	473	35507

(b)

	Normal	Attack
Normal	3895	177
Attack	115	8920

TABLE VI
SUMMARY OF THE PRIMARY AI'S PERFORMANCE

	Accuracy	**MCC**	**UR**
WUSTL-IIoT, Training	99.98%	99.88%	0.19%
WUSTL-IIoT, Test	99.98%	99.86%	0.23%
NSL-KDD, Training	99.30%	98.60%	1.13%
NSL-KDD, Test	99.24%	98.47%	1.18%
UNSW, Training	97.84%	94.98%	1.3%
UNSW, Test	97.77%	94.78%	1.27%

These tables have been brought up to emphasize once again that to develop an XAI in general, only the labels outputted by the primary AI are important while ignoring the true labels of the samples. This is because, in the XAI domain, we care about explaining the primary AI's behavior, not how accurately it has behaved.

The results of these tables based on the metrics including Accuracy, MCC, and UR using (11), (12), and (13) are summarized in Table VI.

Next, we discuss the steps to build the core of TRUST from the output labels from the primary AI. These steps include picking representatives and mode selection.

C. XAI Model: TRUST

The first step is to run the `pickingReps` function using Algorithm 2 on the training set with their predicted labels by the primary AI. We end up with K factors. Afterward, the MI scores between each representative and the class labels have been calculated to determine the top k representatives that have the highest correlation with the class labels and their importance coefficients.

As mentioned before, the choice of k provides a tradeoff between the speed and the performance of the explainer. Large

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:16:13 UTC from IEEE Xplore. Restrictions apply.

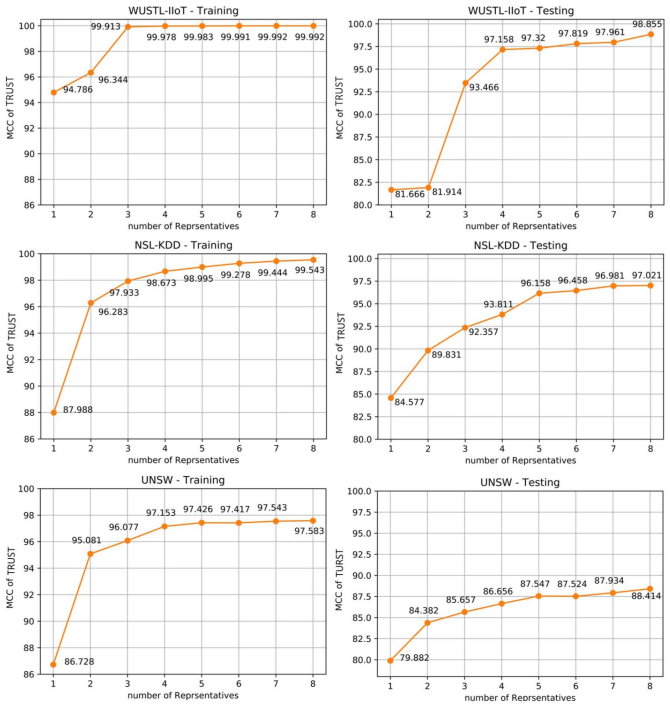


Fig. 4. MCC scores of TRUST XAI on the training sets and the test sets.

Algorithm 5 Grid Mode Selection With $C = 2$

Function: `modeSelectC2`
Input: Representatives of class "1": $\{R_1^1, \ldots, R_k^1\} \in \mathbb{R}^{N_1 \times k}$;
Representatives of class "2": $\{R_1^2, \ldots, R_k^2\} \in \mathbb{R}^{N_2 \times k}$;
Zone $Z \in \mathbb{R}^{2\times 2}$

for each (R_i^1, R_i^2) **do**
  $Max \leftarrow 0$
  for m^1 in Z **do**
    $\text{MMG}_i^1 \leftarrow$ fit R_i^1 to a guassian with m^1 modes
    for m^2 in Z **do**
      $\text{MMG}_i^2 \leftarrow$ fit R_i^2 to a guassian with m^2 modes
      $P_{m^1} \leftarrow p_1(R_i|\text{MMG}_i^1)$ ▷ $R_i = \begin{bmatrix} R_i^1 \\ R_i^2 \end{bmatrix}$
      $P_{m^2} \leftarrow p_2(R_i|\text{MMG}_i^2)$
      $y_{\text{MMG}} \leftarrow P_{m^2} > P_{m^1}$
      $\text{score}_{m^1,m^2} \leftarrow \text{MCC}(y_{\text{MMG}}, \begin{bmatrix} \text{zeros}(N) \\ \text{ones}(N) \end{bmatrix})$
      if $\text{score}_{m^1,m^2} > Max$ **then**
        $Max \leftarrow \text{score}_{m^1,m^2}$
        $M_i^1 \leftarrow m^1$
        $M_i^2 \leftarrow m^2$
      end if
    end for
  end for
end for
return $\{M_1^1, \ldots, M_k^1\}$ & $\{M_1^2, \ldots, M_k^2\}$

k may result in a better performance with a lower speed and *vice versa*. As shown in Fig. 4, just for the sake of experiment, we try k from 1 to 8. Please note that k can be any number between 1 and K. As shown later, we get high MCC scores even with k equal to 4 or 5. Therefore, there is no need to sacrifice the speed or complexity of the model by choosing a large k.

The next step is to estimate the number of modes per representative's pdf. The general case with the C number of classes has been demonstrated in Algorithm 3. In our case, we have only two classes, $C = 2$; therefore, we have only two sets of representatives, $\{R_1^1, \ldots, R_k^1\}$ and $\{R_1^2, \ldots, R_k^2\}$. The modified mode computation algorithms for two classes are shown in Algorithms 5 and 6.

Algorithm 6 Fast Grid Search With $C = 2$

Function: `modeSelectFastC2`
Input: Representatives of class "1": $\{R_1^1, \ldots, R_k^1\} \in \mathbb{R}^{N_1 \times k}$;
Representatives of class "2": $\{R_1^2, \ldots, R_k^2\} \in \mathbb{R}^{N_2 \times k}$;
Zone $Z \in \mathbb{R}^{2\times 2}$

for each (R_i^1, R_i^2) **do**
  Divide zone Z into $z_1, \ldots, z_d$
  $Max \leftarrow 0$
  for each z_j **do**
    $(m^1, m^2) \leftarrow z_j$'s center
    $\text{MMG}_i^1 \leftarrow$ fit R_i^1 to a guassian with m^1 modes
    $\text{MMG}_i^2 \leftarrow$ fit R_i^2 to a guassian with m^2 modes
    $P_{m^1} \leftarrow p_1(R_i|\text{MMG}_i^1)$ ▷ $R_i = \begin{bmatrix} R_i^1 \\ R_i^2 \end{bmatrix}$
    $P_{m^2} \leftarrow p_2(R_i|\text{MMG}_i^2)$
    $y_{\text{MMG}} \leftarrow P_{m^2} > P_{m^1}$
    $\text{score}_{m^1,m^2} \leftarrow \text{MCC}(y_{\text{MMG}}, \begin{bmatrix} \text{zeros}(N) \\ \text{ones}(N) \end{bmatrix})$
    if $\text{score}_{m^1,m^2} > Max$ **then**
      $Max \leftarrow \text{score}_{m^1,m^2}$
      $Z_i \leftarrow z_j$
    end if
  end for
  `modeSelectC2`$(\{R_i^1\}, \{R_i^2\}, Z_i)$
end for
return $\{M_1^1, \ldots, M_k^1\}$ & $\{M_1^2, \ldots, M_k^2\}$

D. Results

The representatives of TRUST XAI are built using the training set. The test samples' factors are calculated by projecting their feature values in the factor space computed using the training set. Then, the same factors chosen as representatives in the training set are chosen for the test samples as well. Afterward, the likelihoods are calculated for the corresponding representatives, and the class with the highest likelihood is explained as the class of that sample.

Fig. 4 shows the MCC results versus the number of representatives k for the training and test sets. Note that MCC values are generally less than accuracy values. The results show TRUST is highly successful in explaining the primary AI's output on unseen data with an average of 90.89% MCC or 98% accuracy.

We have analyzed the performance in terms of time consumption, using the NSL-KDD data set as an example. These experiments were run on a standard laptop with an Intel Core i7 CPU, 16-GB RAM, and Windows OS (no GPU). The steps to build the TRUST model only need to be done once, and after that, the model is reusable to easily explain the labels of future observations. Fig. 5(a) shows the computational time to build the TRUST model (including the factor analysis, picking the representatives, and estimating their density functions) versus the number of representatives k was chosen to build the TRUST model. The time to search for the number of modes is not included in this graph. As expected, by increasing k, it takes more time to build the model since we have more representatives that they require to calculate their density functions.

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:16:13 UTC from IEEE Xplore. Restrictions apply.

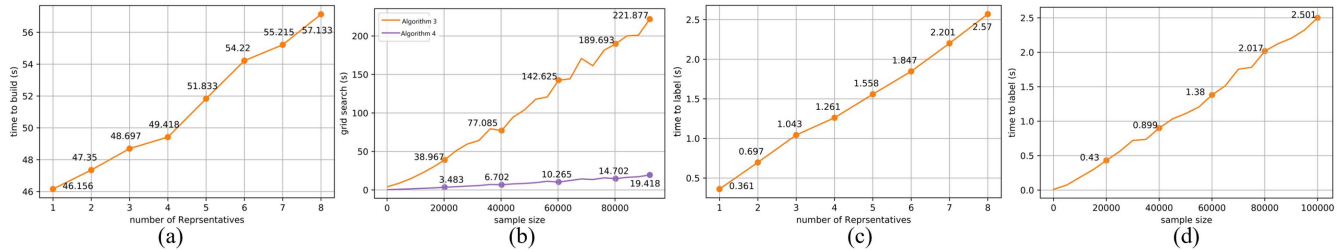


Fig. 5. TRUST's Time consumption of (a) building the core model based on the number of representatives; k, (b) comparing Algorithms 3 and 4; (c) labeling 100 778 samples versus the number of representatives, k; (d) labeling time versus the number of samples, N. In the last two figures, the training set is used because it has a larger number of samples.

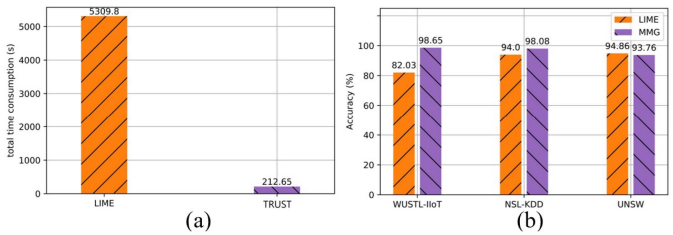


Fig. 6. Explaining 8000 test samples; (a) time consumption of our TRUST model versus LIME, (the TRUST's built time is also included) and (b) performance of TRUST versus LIME.

Algorithms 3 and 4 to determine the number of modes for a representative are compared in Fig. 5(b). The time consumption is calculated versus the number of samples. The search zone is a 20 by 20 grid, and when using Algorithm 4, it was divided into sixteen 5×5 subzones. As seen in this figure, Algorithm 4 speeds up the mode search by more than ten times.

Next, the time for reasoning the labels of test samples by the TRUST model is examined. This step is done after the core of TRUST is built and is ready to label the unseen data as the class with the highest likelihood. First, TRUST labeling time consumption as a function of different numbers of representatives k is plotted in Fig. 5(c). As seen in this figure, the compute time is pretty insignificant. Second, Fig. 5(d) shows the time consumption as a function of the number of samples.

As mentioned before, our model has a significant improvement over the XAI models that require repeating the whole process for every single sample, such as LIME. These repetitions make their model time-consuming and inefficient when dealing with a large number of samples. On the contrary, our technique consists of a core model. Once it is built, it is ready to explain the labels of future unseen data. It is fast and efficient when dealing with a significant amount of unseen incoming data. Fig. 6(a) demonstrates this claim. Note that the TRUST's time of 212.65 s (3.5 min) includes the time to build the model from the 100 778 training samples. The time includes factor analysis, picking eight representatives, running the mode grid search (Algorithm 4) for them, estimating their density functions, and labeling the 8000 test samples. Our model is almost 25 times faster than LIME, which makes a significant difference in real-time applications. Next, we have compared the performance of our model with LIME using the accuracy metric, (12). Here, we used only five representatives in our TRUST model and calculated the accuracy of the test labels. As seen from Fig. 6(b), our model has performed better even with only five representatives, except for UNSW, where LIME has slightly higher accuracy (94.86% versus 93.76%). By increasing the number of representatives, our model would be significantly superior to LIME in all cases. In terms of the difference in explainability methods between TRUST and LIME, we argue that our model produces more straightforward explanations which are easier to understand. LIME is content with local approximations of the primary AI with other simple AI or ML models as a way of explainability and interpretability. Whereas, we model the behavior of the primary AI's output with statistical measures. As statistics are more commonly a part of one's educational background, they are much easier to understand compared to grasping the ideas behind the internal rationale of an AI or ML model.

V. How TRUST Explains the Primary AI Results to the User

This section discusses how the TRUST explainer can explain the labels to a human user simply and interpretably on the NSL-KDD data set as an example. As mentioned before, it is safe to assume that the user of the AI-based security system has a basic knowledge of AI, mathematics, and statistics. Each traffic packet in the NSL-KDD data set is a 40-D vector, but the TRUST explainer would calculate the representatives and reduce each sample to a 4-D vector. This process of dimension reduction is simply explained as follows.

The TRUST explainer modifies the feature values to remove redundancy in the data for a more transparent statistical analysis. By factorizing and removing their interdependencies and projecting them onto orthogonal dimensions, TRUST removes any correlation in the data. The transformed feature values are the factors introduced before. Then, utilizing a correlation metric (here, MI), the most revealing factors determining the class labels are chosen as representatives. Afterward, TRUST models the statistical behavior of the representatives and estimates their density functions with unique multimodal distributions.

As mentioned before, statistics are more easily understood than the AI or ML models used as the XAI. For elaboration, we have shown an example in Fig. 7. It is trivial for the user to understand the likelihood differences of a new sample belonging to the pdfs of each class's representative. In this example, the likelihood of belonging to the attack class is higher.

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:16:13 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
LOG LIKELIHOOD. (a) LOG LIKELIHOOD OF BELONGING TO ATTACK CLASS. (b) LOG LIKELIHOOD OF BELONGING TO NORMAL CLASS. (c) COMPARISON OF LOG LIKELIHOOD. (d) TOTAL LOG LIKELIHOOD

(a)

	1st Rep.	2nd Rep.	3rd Rep.	4th Rep.
Attack samples	5.51	5.46	4.40	4.27
	1.56	-2.46	-1.19	1.54
	-1.79	-2.06	0.68	-3.06
Normal samples	-3.61	-2.09	-2.30	-5.21
	-6.99	-2.16	-3.12	-2.99
	-3.56	-2.19	-10.09	-4.72

(b)

	1st Rep.	2nd Rep.	3rd Rep.	4th Rep.
Attack samples	-0.30	1.87	-0.04	-0.86
	0.08	-4.92	-1.11	-0.89
	-2.52	-2.16	-0.31	-6.18
Normal samples	-3.68	1.86	-2.11	-5.16
	-3.25	1.83	-2.76	-0.94
	-3.02	1.75	-2.41	-0.90

(c)

1st Rep.	2nd Rep.	3rd Rep.	4th Rep.
5.51 > -0.30	5.46 > 1.87	4.40 > -0.04	4.27 > -0.86
1.56 > 0.08	-2.46 > -4.92	-1.19 < -1.11	1.54 > -0.89
-1.79 > -2.52	-2.06 > -2.16	0.68 > -0.31	-3.06 > -6.18
-3.61 > -3.68	-2.09 < 1.86	-2.30 < -2.11	-5.21 < -5.16
-6.99 < -3.25	-2.16 < 1.83	-3.12 < -2.76	-2.99 < -0.94
-3.56 < -3.02	-2.19 < 1.75	-10.09 < -2.41	-4.72 < -0.90

(d)

Total
19.64> 0.67
-0.55>-6.84
-6.23>-11.17
-13.21<-9.09
-15.26<-5.12
-20.56<-4.58

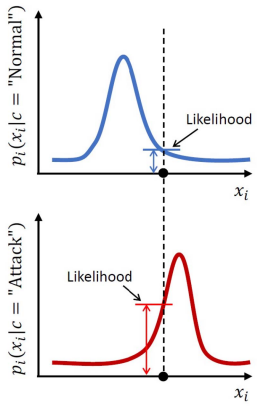


Fig. 7. Example of pdf curves for the *i*th representative.

To show the explanation process of TRUST, we randomly chose six samples (three attacks and three normals) from the test set. The log likelihoods of belonging to each class using individual representatives based on (6) are computed. The results of each instance belonging to the attack class and belonging to the normal class are shown in Tables VII(a) and (b), respectively. After an elementwise comparison of the values in these two matrices, the class that has a higher likelihood is marked in Table VII(c). Note that not all the representatives give us the right class all the time. For example, the 1st representative makes mistakes with the 4th sample. Similarly, the 3rd representative makes mistakes with the 2nd sample. However, when all four representatives are used to calculate the overall log likelihood, as shown in Table VII(d), the labeled class is correctly explained for all six instances.

VI. CONCLUSION AND FUTURE DIRECTION

XAI has attracted extensive attention recent years and has been treated as a necessary trend toward the next generation of AI. However, there is a significant lack of research work in this domain, specifically for numerical applications (compared to image applications). Meanwhile, several critical applications, such as cybersecurity and IIoT are highly dependent on numerical data. The available work in the literature falls short from several aspects, such as tradeoffs among performance, speed, reusability, and complexity, as discussed in this article. Our proposed TRUST XAI is a breakthrough in this domain in integrating transparency, being independent, and comparability (in a statistical aspect) into AI-based systems. Here, we take the lead to use statistical principles to build XAI models that can accurately estimate the distributions of the AI's outputs and calculate the likelihood of new samples belonging to each class. Moreover, we have proven the superiority of our model to a currently popular XAI model, LIME. Our results show that our model is 25 times faster and more accurate than LIME making our model a great candidate for real-time and critical applications.

Despite TRUST's superior performance, it has some limitations as well. Due to using information gain in picking the representatives, TRUST might overfit to the training set. This would lead to poor performance on unseen data. On the other hand, if the Gaussian assumption cannot be made or the probability distribution of data changes, the output of TRUST would not be reliable. Also, the assumption of samples being drawn independently is very important.

As a future direction, for the selection of representatives, we can take into account other metrics such as the Gini index. In addition, we can extend the model to consider other possible probability distributions (e.g., Poisson). Also, removing the independence assumption requires approximating the joint probabilities of each pair of features.

REFERENCES

[1] Gartner. *Gartner Says Nearly Half of CIOs Are Planning to Deploy Artificial Intelligence*. (2018). Accessed: Oct. 22, 2021. [Online]. Available: https://tinyurl.com/5xupkeur

[2] Webroot. *Game Changers: AI and Machine Learning in Cybersecurity*. (2017). Accessed: Oct. 22, 2021. [Online]. Available: https://tinyurl.com/49sf724t

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:16:13 UTC from IEEE Xplore. Restrictions apply.

[3] I. Bartoletti, "AI in healthcare: Ethical and privacy challenges," in *Artificial Intelligence in Medicine*. Cham, Switzerland: Springer Int., 2019, pp. 7–10.

[4] A. Holzinger, C. Biemann, C. S. Pattichis, and D. B. Kell, "What do we need to build explainable AI systems for the medical domain?" 2017. [Online]. Available: arXiv:1712.09923.

[5] C. Oxborough and E. Cameron. *Explainable AI: Driving Business Value Through Greater Understanding*. (2018). Accessed: Oct. 22, 2021. [Online]. Available: https://www.pwc.co.uk/xai

[6] T. Miller, "Explanation in artificial intelligence: Insights from the social sciences," *Artif. Intell.*, vol. 267, pp. 1–38, Feb. 2019.

[7] A. Adadi and M. Berrada, "Peeking inside the black-box: A survey on explainable artificial intelligence (XAI)," *IEEE Access*, vol. 6, pp. 52138–52160, 2018.

[8] Z. Liu, B. Xiao, M. Alrabeiah, K. Wang, and J. Chen, "Single image dehazing with a generic model-agnostic convolutional neural network," *IEEE Signal Process. Lett.*, vol. 26, no. 6, pp. 833–837, Jun. 2019.

[9] M. Zolanvari, M. A. Teixeira, L. Gupta, K. M. Khan, and R. Jain, "Machine learning-based network vulnerability analysis of Industrial Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6822–6834, Aug. 2019.

[10] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Security Defense Appl.*, Ottawa, ON, Canada, 2009, pp. 1–6.

[11] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, 2015, pp. 1–6.

[12] A. Gilchrist, *Industry 4.0: The Industrial Internet of Things*. Heidelberg, Germany: Springer, 2016.

[13] K. A. da Costa, J. P. Papa, C. O. Lisboa, R. Munoz, and V. H. C. de Albuquerque, "Internet of Things: A survey on machine learning-based intrusion detection approaches," *Comput. Netw.*, vol. 151, pp. 147–157, Mar. 2019.

[14] C. Alcaraz, *Security and Privacy Trends in The Industrial Internet of Things*, 1st ed. Cham, Switzerland: Springer Int., 2019.

[15] K. Gade, S. C. Geyik, K. Kenthapadi, V. Mithal, and A. Taly, "Explainable AI in industry," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min. (KDD)*, New York, NY, USA, 2019, pp. 3203–3204.

[16] S. Mahamood, "Explainable artificial intelligence and its potential within industry," in *Proc. 1st Workshop Interact. Nat. Lang. Technol. Explainable Artif. Intell. (NL4XAI)*, vol. 58, 2019, pp. 1–2.

[17] I. Siniosoglou, P. Radoglou-Grammatikis, G. Efstathopoulos, P. Fouliras, and P. Sarigiannidis, "A unified deep learning anomaly detection and classification approach for smart grid environments," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1137–1151, Jun. 2021.

[18] W. Guo, "Explainable artificial intelligence for 6G: Improving trust between human and machine," *IEEE Commun. Mag.*, vol. 58, no. 6, pp. 39–45, Jun. 2020.

[19] I. Garcia-Magarino, R. Muttukrishnan, and J. Lloret, "Human-centric AI for trustworthy IoT systems with explainable multilayer perceptrons," *IEEE Access*, vol. 7, pp. 125562–125574, 2019.

[20] M. A. Ahmad, C. Eckert, and A. Teredesai, "Interpretable machine learning in healthcare," in *Proc. ACM Int. Conf. Bioinformat. Comput. Biol. Health Informat. (BCB)*, New York, NY, USA, 2018, pp. 559–560.

[21] E. Tjoa and C. Guan, "A survey on explainable artificial intelligence (XAI): Toward medical XAI," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Oct. 20, 2020, doi: 10.1109/tnnls.2020.3027314.

[22] R. Guidotti, A. Monreale, F. Turini, D. Pedreschi, and F. Giannotti, "A survey of methods for explaining black box models," *ACM Comput. Surveys*, vol. 51, no. 5, pp. 1–93, 2018.

[23] E. Daglarli, *Explainable Artificial Intelligence (XAI) Approaches and Deep Meta-Learning Models for Cyber-Physical Systems*. London, U.K.: IGI Global, 2021, pp. 42–67.

[24] N. A. Seresht, Q. Liu, and Y. Miao, "An explainable intelligence model for security event analysis," in *Proc. Aust. Joint Conf. Artif. Intell.*, 2019, pp. 315–327.

[25] S. M. Mathews, "Explainable artificial intelligence applications in NLP, biomedical, and malware classification: A literature review," in *Proc. Intell. Comput. Conf.*, 2019, pp. 1269–1292.

[26] A. B. Arrieta *et al.*, "Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Inf. Fusion*, vol. 58, pp. 82–115, Jun. 2020.

[27] C. Molnar. *Interpretable Machine Learning. A Guide for Making Black Box Models Explainable*. (2018). Accessed: Oct. 22, 2021. [Online]. Available: https://christophm.github.io/interpretable-ml-book/

[28] J. Li *et al.*, "Feature selection: A data perspective," *ACM Comput. Surveys*, vol. 15, no. 6, pp. 1–94, 2017.

[29] M. Wang, K. Zheng, Y. Yang, and X. Wang, "An explainable machine learning framework for intrusion detection systems," *IEEE Access*, vol. 8, pp. 73127–73141, 2020.

[30] S. M. Lundberg and S. I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds. Red Hook, NY, USA: Curran Assoc., Inc., 2017, pp. 4765–4774.

[31] K. Amarasinghe, K. Kenney, and M. Manic, "Toward explainable deep neural network based anomaly detection," in *Proc. 11th IEEE Int. Conf. Human Syst. Interact. (HSI)*, 2018, pp. 311–317.

[32] O. Sagi and L. Rokach, "Explainable decision forest: Transforming a decision forest into an interpretable tree," *Inf. Fusion*, vol. 61, pp. 124–138, Sep. 2020.

[33] D. L. Marino, C. S. Wikramasinghe, and M. Manic, "An adversarial approach for explainable AI in intrusion detection systems," in *Proc. 44th Annu. Conf. IEEE Ind. Electron. Soc. (IECON)*, Washington, DC, USA, 2018, pp. 3237–3243.

[34] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you? Explaining the predictions of any classifier," in *Proc. ACM SIGKDD Int. Conf. Knowl. Disc. Data Min. (KDD)*, San Francisco, CA, USA, 2016, pp. 1135–1144.

[35] M. Zafar and N. Khan, "DLIME: A deterministic local interpretable model-agnostic explanations approach for computer-aided diagnosis systems," 2019. [Online]. Available: arXiv:1906.10263.

[36] J. Rabold, H. Deininger, M. Siebers, and U. Schmid, "Enriching visual with verbal explanations for relational concepts—Combining lime with aleph," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Disc. Databases*, 2019, pp. 180–192.

[37] M. Ribeiro, S. Singh, and C. Guestrin, "Anchors: High-precision model-agnostic explanations," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, 2018, pp. 1527–1535.

[38] J. Pages, *Multiple Factor Analysis by Example Using R*. New York, NY, USA: Chapman and Hall, 2015.

[39] A. Kassambara, *Practical Guide To Principal Component Methods in R*. New York, NY, USA: STHDA, 2017.

[40] R. Jain, *The Art of Computer Systems Performance Analysis*. Hoboken, NJ, USA: Wiley-Intersci., 1991.

[41] S. Maxwell, H. Delaney, and K. Kelley, *Designing Experiments and Analyzing Data: A Model Comparison Perspective*, 3rd ed. London, U.K.: Routledge, 2017.

[42] T. Cover and J. Thomas, *Elements of Information Theory*, 2nd ed. Chichester, U.K.: Wiley, 2006.

[43] S. K. Ng, T. Krishnan, and G. J. McLachlan, "The EM algorithm," in *Handbook of Computational Statistics*. Heidelberg, Germany: Springer, 2012, pp. 139–172.

[44] M. Zolanvari, M. A. Teixeira, and R. Jain, "Effect of imbalanced datasets on security of industrial IoT using machine learning," in *Proc. IEEE Int. Conf. Intell. Security Informat. (ISI)*, 2018, pp. 112–117.

[45] M. A. Teixeira, T. Salman, M. Zolanvari, R. Jain, N. Meskin, and M. Samaka, "SCADA system testbed for cybersecurity research using machine learning approach," *Future Internet*, vol. 10, no. 8, pp. 1–15, 2018.

[46] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Jan. 2011.

Authorized licensed use limited to: North South University. Downloaded on October 30,2026 at 17:16:13 UTC from IEEE Xplore. Restrictions apply.